

Milan

Application Q&A — 28 judge questions, answered from the code.

Problem statement	SIH26043 — Government of Jharkhand, Disaster Management (Software)
Edition	Smart India Hackathon 2026
One-line pitch	Converts a citizen's verified local problem into a time-bound, routed research assignment for a
Stack (one cut)	Next.js 15 + React 19 · Tailwind v4 + shadcn/ui · Drizzle + Zod · Supabase Postgres 17 (pgvec)
What is not here	No blockchain. No separate vector DB. No Kafka. No fine-tuned models. No anonymous read.
Demo at	/demo — health strip, six-button console, real production server actions, no terminal.

The five sentences to memorise

1. Disaster management is mostly mitigation. We are a mitigation pipeline that runs in peacetime, and every challenge carries an explicit hazard linkage.
2. CPGRAMS routes complaints to officers. We route unsolved problems to labs, with a clock. When something is a grievance, we forward it to CPGRAMS.
3. Discovery is never luck. Every problem is pushed to matched departments, and every state has an SLA with an automatic escalation.
4. Universities are not doing us a favour — 200,000 Indian students invent a fake final-year project every year. We give them real ones.
5. We do not stop people from sharing work. We make it impossible to erase who did it.

Problem & Concept

Q. 1. This is filed under Disaster Management, but you're talking about agriculture, water, education. Explain.

A.

Disaster management in India is overwhelmingly mitigation and preparedness, not response. The National Disaster Management Plan and the Sendai Framework both put the work **before** the event — reducing risk, building capacity, identifying vulnerabilities. An embankment that cracks in the dry season, a watershed that has lost its cover, a school that floods every monsoon, a public-health signal that precedes an epidemic — these are all disaster-management problems expressed through the sector they touch. Milan does not replace the agriculture, water or education departments; it routes unsolved problems from those sectors to a research lab so the fix exists before the disaster. Every challenge in Milan carries an explicit NDMA hazard linkage (FLOOD, DROUGHT, LANDSLIDE, HEATWAVE, MINING_SUBSIDENCE, EPIDEMIC, FOREST_FIRE, or NONE) and that linkage is a weighted term in the priority score. Agriculture, water and education problems that **do** carry a hazard are exactly the work DM exists to fund; problems without a hazard link still flow through, because what a citizen sees as "education" the system sees as "school vulnerability". Filed under DM, because the platform's organising idea is risk reduction.

Q. 2. How is Milan different from CPGRAMS or JharSewa? Why build another portal?

A.

CPGRAMS and JharSewa route complaints with a **known fix** to an accountable officer. Milan routes **unsolved problems** to a lab, with a clock. Concretely: when a citizen reports a broken handpump with a sanctioned PHED contractor, that is a grievance — S1 detects it, Milan hands it off to the right grievance portal, and the citizen sees the exact JSON payload Milan would have POSTed and the reference number it would have received. When a citizen reports a recurring monsoon flood in a ward that has no engineering answer yet, CPGRAMS cannot help — there is no officer to complain to, because nobody knows the fix. Milan treats that as a research question, frames it, scores it, routes it to the university department whose capability graph best matches, opens a 7-day claim clock, and never lets it silently die. We are not a fourth portal. We are the pipeline that catches what grievance portals structurally cannot, and we hand back to them the things that **are** grievances, with full traceability.

Q. 3. What is the actual core problem you are solving?

A.

Two compounding failures in Indian civic problem-solving. First, discovery is luck: most unsolved local problems never reach a researcher who can solve them, because there is no pipeline. Second, when a problem is picked up, contribution is invisible: a citizen's testimony, a student's fieldwork, a panchayat's local knowledge, a professor's guidance — none of it has a durable, verifiable record. So the same problems get re-surveyed badly every five years by different consultants who never know what came before. Milan fixes both: it pushes every verified problem to a matched university department with an SLA clock, and it writes every contribution into a hash-chained credit ledger that nobody can erase. The product is the join of those two fixes — that join is what does not exist today.

Q. 4. Why would this work in Jharkhand specifically? Is it not too rural for a tech platform?

A.

Jharkhand is the right size and the right shape. It is small enough (24 districts, ~33 million people) that the entire state is in scope for one deployment, but large and varied enough (tribal heartland, mining districts, Kharif-dependent agriculture, recurring flood and drought corridors) that the platform is exercised across every hazard and domain. The seed already covers all 24 districts and 263 blocks against real Jharkhand boundary geometry. "Too rural" is the exact problem the design addresses: intake works on a 320px phone in Hindi, with offline drafts, an EXIF-stripping upload, a tap-to-blur privacy step, and an IVR/WhatsApp seam reserved for the next cut. The citizen never has to log in to confirm impact (a signed link, 90-day TTL, works on any device). What is *rural* in Jharkhand is also what is digitally underserved everywhere — solving it here generalises. Finally, the tribal languages are not a blocker to a typed or voice report: P0 transcribes voice notes (Groq whisper-large-v3), and the citizen's own words render at the same size as any English copy, never behind a toggle.

Q. 5. How is Milan different from SIH itself?

A.

SIH is a once-a-year competition that produces a working prototype and then moves on. Milan is the after-SIH plumbing: a permanent registry of real problems, a permanent record of who solved them, and a permanent ladder that ensures the unsolved ones do not die in a drawer. SIH evaluates a team; Milan evaluates an outcome, against the citizen's own confirmation. SIH ends on a demo day; Milan is the demo day becoming a year-round pipeline, fed by the same citizen reports, scored the same way, claimed the same way, and the impact counter only moves when a real person says "yes, this changed something in my village". We are not replacing SIH; we are what SIH's winning teams need to exist inside, long after the hackathon is over.

AI & Technology

Q. 6. Walk us through how the AI works.

A.

Six stages, all in ``lib/ai/pipeline.ts``, with every model call writing an ``ai_runs`` receipt. **P0 (Language)** transcribes any attached voice note and produces an English working copy of the report — the citizen's own words are never overwritten. **S1 (Safety/Triage)** classifies the report as unsafe, grievance, low-confidence, or a research problem. Unsafe content (confidence ≥ 0.60) is rejected with a helpline; grievances with textual evidence are forwarded to CPGRAMS/JharSewa form; everything below 0.60 confidence is held for a human at ``/admin/triage`` rather than auto-routed. **S2 (Domain/Hazard)** picks one of ten domains and one of eight NDMA hazards with a strength, and kicks off S5 in parallel. **S3 (Clustering)** does pgvector cosine matching against existing challenges: ≥ 0.86 auto-merges, 0.72–0.86 is adjudicated by a single yes/no model call, < 0.72 is distinct. Duplicates are *signal* — the older report is preserved as the survivor, both reporters are credited, a corroboration row increments the count. **S4 (Priority score)** is a pure TypeScript function in ``packages/scoring`` — zero model calls, seven weighted terms that sum to exactly 1.00, all rendered visibly on the public challenge page. **S5 (Routing)** ranks institutions against the Institutional Capability Graph on five signals (semantic fit 0.45, specialisation 0.20, distance 0.15, declared capacity 0.12, track record 0.08) and writes the top 3 distinct organisations. If severity ≥ 0.70 , those three offers are written with ``notified_at = null`` — nothing is sent until a District Collector confirms at ``/gov/gate``.

Q. 7. What models are you using? Is this just ChatGPT wrapped in a UI?

A.

It is not a wrapper, and the distinction matters because we have to be right at the last mile, not at the prompt. The live chain is **Gemini Flash** → **Groq (openai/gpt-oss-120b)** → **deterministic rules**, tried in order, with a deliberately-appended rules fallback that means a provider outage is a fallback level, not an outage. Embeddings are 768-d and live in pgvector. We do **not** fine-tune — there is no labelled data and no GPU budget — but every human override at ``/admin/triage`` or ``/gov/gate`` writes a row to ``training_corrections``, which is the labelled data for the next embedding kNN prior. So the system "learns" from human corrections without ever being trusted to act on its own uncertainties. We are also explicit about which decisions the model **cannot** make: the priority score (S4) is a pure function with no model calls reachable from it at all, the routing bar (85/100) is deterministic, the human gate at severity ≥ 0.70 is structural in code, and the routing reason sentences are post-validated by ``guardReason()`` which rejects any number not in the supplied facts. The model proposes; code decides. That's the invariant.

Q. 8. How does the routing engine work? How does it know which university to send a problem to?

A.

Each institution publishes an **Institutional Capability Graph** — a set of capability rows (e.g. "hydrology + flood modelling + Koshi basin", "rural water supply + Santhal Pargana", "soil stabilisation + laterite") with department, tags, declared open capacity for the current window, location, and a track record. S5 computes a match score per institution: $0.45 \times \text{cosine}(\text{challenge embedding}, \text{capability embedding}) + 0.20 \times \text{Jaccard}(\text{specialisation tags}, \text{domain+hazard keyword expansion}) + 0.15 \times \exp(-\text{km} / 250) + 0.12 \times \text{declared capacity (capped at 5)} + 0.08 \times \text{Laplace-smoothed track record}$. Ranked, then the top **3 distinct organisations** — never two labs at one college, because a shortlist of three departments in one place is not a shortlist. The model is then asked to write one sentence explaining the top three contributing terms for each of the three offers, but it is handed only those three terms — not the challenge text, not the institution facts. ``guardReason()`` then rejects any sentence containing a number that was not in the supplied facts. A rejected sentence falls back to a blunt template over the same three facts. This is a structural guarantee in code, not a prompt instruction.

Q. 9. What happens when 200 people report the same broken water pipeline?

A.

Two layers, both running on every submission. **S3 (clustering)** treats it as one event, not 200. The first report becomes the survivor and keeps its tracking ID; reports 2–200 are marked ``MERGED`` (terminal) with a ``CORROBORATOR`` credit edge onto the survivor — nobody is discarded, nobody is double-counted, the older report is always preserved (older ID wins on a same-instant tie). Each merged report also writes a ``CREDIT_EDGE`` ledger entry that says "nothing was discarded". The survivor's ``corroboration_count`` increments; if it crosses three distinct reporters across two or more blocks, S3 also spawns a ``BLOCK_SYSTEMIC`` parent challenge that is itself scorable and routable. **Anti-brigading** runs on top: one account, one corroboration per challenge (unique index), corroboration weight decays with distance from the incident (full weight within 15 km, exponential decay beyond, never zero), a 5-per-hour submission rate limit per actor, and a decaying trust score (earned only at ``CITIZEN_VERIFIED``, lost on ``REJECTED_UNSAFE``, decayed nightly toward 0.50) that weights the corroboration term in S4. So 200 reports from 200 real neighbours, all within 15 km, all with high trust, genuinely do raise the priority — and 200 reports from one device fingerprint are exactly one report, with the fingerprint flagged and zeroed for transparency.

Q. 10. How do you handle submissions in Santhali, Ho, or Mundari?

A.

Honestly, in the first cut we cover Hindi and English end-to-end, plus one Santhali sample for the demo. We say so on a slide rather than pretending. The path for the other languages is engineered: P0 uses Groq ``whisper-large-v3`` for voice transcription, which supports Santhali, Ho and Mundari in addition to the major Indic languages; for text, the S1/S2 prompts include few-shot examples in the target script. What we do **not** do, on purpose, is hide the citizen's own words behind a translation. The challenge page renders the original text at the same size and weight as the English working copy, side by side, and the database itself documents this with a column comment in migration 0006 — that is invariant 6. If a translation fails, ``body_en`` is left null rather than filled with the original; rendering the citizen's Hindi under a heading that says "English working copy" would be a small lie on a page whose whole argument is that nothing is hidden. The roadmap is: add a curated few-shot per language with a native speaker, expand transcription coverage, and let every human correction in ``training_corrections`` improve the next automatic answer.

Q. 11. What is your tech stack?

A.

One Next.js 15 deployable, one Postgres 17 database, one outbox table standing in for a queue, one AI provider chain. Concretely: **Next.js 15 (App Router, RSC) + React 19 + Tailwind v4 + shadcn/ui** on the front end; **Leaflet + Protomaps (PMTiles, offline-ready)** for maps; **Recharts** for the stats page; **Better Auth** with a hand-rolled OTP layer on top; **Drizzle ORM + Zod** for the schema and validation; **Supabase PostgreSQL 17 with pgvector (HNSW), FTS, and pg_trgm** for the database; **Supabase Storage (or MinIO offline)** keyed by content hash; **Gemini Flash → Groq → deterministic rules** for the LLM chain; **SSE** (native ReadableStream) for the pipeline trace; **Vercel + Vercel Cron** for hosting and scheduling; **GitHub Actions** for CI; **pnpm** workspaces. Deliberately excluded, with reasons: **blockchain** (a SHA-256 chain plus a public timestamp gives the same non-repudiation at zero cost and zero latency), **a separate vector database** (pgvector with HNSW scales to millions of rows), **Kafka** (a transactional outbox covers every event at this state scale), **fine-tuned models** (no labelled data, no GPU budget — few-shot + embedding kNN prior, declared honestly), and **a separate API and inference service** (one deployable, every module in its own folder so it splits out later without a rewrite).

Q. 12. How do you ensure AI decisions are fair and not a black box?

A.

Three layers. **Layer 1 — what the model cannot do.** S4 (priority score) is a pure TypeScript function with zero model calls reachable from it. The human gate at severity ≥ 0.70 is structural in code: ``persistRoutes()`` writes the shortlist with ``notified_at = null``, and nothing is sent until a District Collector confirms at ``/gov/gate``. The 85/100 routing bar is deterministic — anything below is ``PARKED``, not "low priority routed". **Layer 2 — what the model is allowed to say.** ``guardReason()`` rejects any routing reason sentence that contains a number not present in the facts the model was given. A rejected sentence falls back to a template. The model never sees the challenge text when writing routing reasons — only the top three contributing terms — so it cannot leak demographic details into the sentence. **Layer 3 — what the public can see.** Every score is rendered on the public challenge page with the full breakdown — each term's raw value, normalised value, weight and contribution, the arithmetic visible and clickable. That is invariant 10. Every model call writes a receipt to ``ai_runs`` (provider, model, fallback level, confidence, latency, input hash) viewable at ``/admin/ai-runs``. Every human override is logged to ``audit_log`` and to ``training_corrections``, so the system's behaviour under override is itself auditable. We are not claiming the model is fair; we are claiming the system around the model constrains the model's room to be unfair, and shows its work.

Loopholes & Risks

Q. 13. What if no university ever claims a challenge?

A.

There is a four-rung ladder, all on a clock. **+7 days (WIDEN)**: S5 is re-run and five more institutions are offered the challenge — the next-five-best from the original graph, with the shortlist reason rendered the same way. **+14 days (OPEN_ALL)**: the challenge is visible and claimable in every `/hei/challenge-bank` in the state, and the district officer is notified. **+21 days (BREACH)**: `sla_breached_at` is stamped, the challenge is listed on the public `/bounties` board, and the DC + admins are notified. **+45 days (GRAND_CHALLENGE)**: the challenge joins the annual Jharkhand Grand Challenges set, and an `ANNUAL_REVIEW` deadline (365 days, never compressed by Emergency Mode) keeps it visible. Importantly, none of this is a cron job someone remembered to write. Every non-terminal state carries a durable `sla_deadlines` row, opened in the same transaction as the state change itself, and a CI query fails the build if any challenge anywhere is without one. `tests/invariant.test.ts` is a required CI check that returns zero orphans. "No one claimed it" is a state, and that state has consequences.

Q. 14. What if a team takes a challenge and then abandons it midway?

A.

Two further ladders, both on `projects.last_activity_at` (not the claim date — the last thing the team actually did). **Ladder 2 — silence on the proposal**. `CLAIMED` opens `PROPOSAL_DUE` at +14 days: the lead and the HOD are nudged, then re-nudged at +21 days. At +21 the claim is **released** — the project row is marked `RELEASED_UNDELIVERED` (never deleted), the offer routes clear back to `ROUTED`, and the prior team's project, credit edges, and any partial work are preserved and attributed permanently. "We do not stop people from sharing work. We make it impossible to erase who did it" applies to a team that failed to deliver as much as to one that succeeded. **Ladder 3 — silence mid-project**. `PROPOSAL_APPROVED`/`IN_RESEARCH` opens `SILENT_30` and `SILENT_45`. At 30 days the project is publicly flagged `AT_RISK` and the mentor is nudged. At 45 days **fork rights open** — another team may fork the work, with the original team's project and credit edges kept intact and visible on the fork.

Q. 15. What if someone submits fake or malicious reports to game the system?

A.

Three concentric defences. **Input gating**: a deterministic offline moderation pre-filter (`lib/moderation/blocklist.ts`) catches profanity, low-effort/troll phrases, character-spam and repeated-word heuristics before the request hits the model; a 5-per-hour rate limit per actor; one account, one corroboration per challenge (unique index); corroboration weight decays with distance from the incident (full weight within 15 km, exponential decay beyond, never zero); more than 3 corroborations from one device fingerprint or more than 25 in one hour flags that fingerprint and zeroes its weight (never deletes it — it stays visible as evidence someone tried); corroborations from more than 150 km away are flagged as down-weighted. **Output correction**: S2's classification and S5's routing are reviewed by a human at `/admin/triage` and `/gov/gate`; overrides are written to `training_corrections` and become labelled data. **Trust as a structural disincentive**: a `REJECTED_UNSAFE` event costs the reporter -0.20 in trust, deliberately larger than five good events combined, so brigading a false narrative and then reporting something real does not net out to zero. Trust decays nightly toward a 0.50 baseline (good standing must be renewed; bad standing is forgivable). Finally, the impact counter — the only number anyone outside the system cares about — moves only on `CITIZEN_VERIFIED`, i.e. when the *citizen* says the problem was real and the fix worked. You can game the queue, but you cannot game the citizen's confirmation.

Q. 16. What if a company steals a university's research and publishes it as their own?

A.

Two layers of defence. **Defensive publication as prior art.** Publication writes a ledger entry committing to the file's own SHA-256 at a timestamp. That does not stop a patent being filed — nothing can — but it makes the work prior art, which is what stops one being granted over it. The prior-art panel on `/artifacts/[id]` says exactly that, and the ledger entry is the timestamp a patent examiner will see. **Access logging on restricted files.** A CC-BY file is open to anyone with attribution; a RESTRICTED file is read only through a verified identity with a stated purpose, granted by the project lead, and every actual download writes an `access_log` row and an `ACCESS` ledger entry visible to the team and to the originating citizen. There is no anonymous read. If a company is later found using the work without permission, the access log shows who, when, and under what stated purpose. The MoU is generated from a template, SHA-256 hashed, and the hash is appended to the ledger; e-signature and payment rails are declared stubs, and the document says so on its own face. IP dispute adjudication is also a declared stub — but the evidence the dispute would be decided on is in place and verifiable.

Q. 17. What if the AI misclassifies a serious problem — say, sends a flood-risk report to the wrong department?

A.

Three things make that rare, and one thing makes it reversible. **Rare, because the S2 classifier is constrained.** S2 is seeded with an embedding KNN prior over already-classified challenges — the closest thing this system has to "learning" without a fine-tune. Every human correction lands in `training_corrections` and improves the next prior. S2 also picks from a fixed taxonomy of ten domains and eight NDMA hazards, so the misclassification modes are enumerable. **Rare, because the routing reason is guardrailed.** `guardReason()` rejects any sentence containing a number not in the supplied facts — the model cannot invent "five villages at risk" if only two are in the input. **Rare, because the human gate stands in front of consequential routing.** At severity ≥ 0.70 , `notified_at` is null until a District Collector confirms at `/gov/gate` — the wrong department would not be notified without a human's signature. **Reversible, because the state machine is explicit.** Every state change writes a `STATE_CHANGE` ledger entry and an `audit_log` row; admins can reroute from `/admin/routing` with a mandatory written reason, and from `/admin/challenges` with a state override that demands a reason. The original state and the override are both on the chain. "Misclassified" is detectable, attributable, and fixable — not silently absorbed.

Q. 18. What if a citizen's report contains someone's face or personal data in a photo?

A.

Stronger than "anonymised after upload" — the unblurred original never leaves the device. The submit wizard has a photo step that opens a canvas mosaic; the citizen taps to place a blur box over faces, number plates, or anything else they do not want public, and only the blurred bytes are ever uploaded. `challenge_media.faces_blurred` records what the citizen actually blurred, so the audit trail is honest about whether the citizen did the work. The server then strips EXIF and GPS metadata from the processed image (`lib/media/upload.ts`, using `sharp`) and re-keys the object by the SHA-256 of the processed bytes. Automatic face and number-plate detection is a **declared stub** — we do not pretend to have it, and we do not need it, because the citizen-driven blur is a stronger privacy claim than an automated one (an automated detector that misses a face in shadow is worse than a human who can see the photo they took). Storage itself is keyed by content hash, so a citizen who deletes the original from their phone after upload still has the same hash on the platform; conversely, the platform holds only the blurred bytes, by construction.

Q. 19. What stops the platform from becoming a ghost town after SIH?

A.

Three structural reasons it should not, and one acknowledgement. **Reason 1 — the supply is the demand.** Every citizen report is a real final-year project. We are not "getting universities to participate"; we are giving them work they were already going to invent badly, except this time it is real. 200,000 Indian students invent a fake final-year project every year. We give them real ones, with a real citizen, a real credit chain, and a real publication path. **Reason 2 — the citizen confirmation is the only number that moves.** The product never converges to "shut up and count". Confirmed, partly confirmed, and claimed-not-confirmed are three separate counters, never summed. A CSR report built on this data is one an auditor can defend. **Reason 3 — the platform has multiple constituencies with different retention curves.** Citizens return to confirm impact (a 90-day TTL signed link, no login). District Collectors use it because the SLA board is the only place they see their overdue items with a derivation. Industry uses it because the CSR export is the only one that won't collapse under audit. HEI members use it because their students get real credit, by name, on a public chain. **Acknowledgement:** with no sustained outreach, an empty portal is a real risk. The honest answer is: we do not stop the ghost-town risk by ourselves; we stop it by being the thing the Department of Disaster Management already wants (a mitigation pipeline), the thing CSR teams already need (defensible impact data), and the thing universities already do (real capstone projects). One product, three buyers, one shared ledger.

Adoption & Actors

Q. 20. Why would a citizen bother submitting? Most people don't trust these portals.

A.

Three reasons, in increasing order of importance. **It is easier than a tweet.** Six steps on a 320px phone, in Hindi, with offline drafts, an EXIF-stripping upload, a tap-to-blur privacy step, and a tracking ID sayable over a phone ("JH-2026-GUM-0001") so the citizen can call back and ask. **They get a real answer, not a reference number.** A grievance gets forwarded to CPGRAMS/JharSewa, with the exact JSON payload Milan would have POSTed shown on the public page. A research problem gets routed to a real university department, with a name and a clock. **The citizen's own words stay on the page, and the citizen is on the team.** A research problem that gets claimed credits the reporter as "Domain Informant" on the project by default; their name is in the citation string before the institution, in the author position. **And only the citizen moves the impact counter.** When someone eventually claims the problem is fixed, the only number that moves anywhere in Milan is the one that moves when the citizen says yes. Trust is built by giving people the one thing most portals do not: a final say, with their name on it.

Q. 21. Why would a professor or HEI engage? They're already busy.

A.

Because the work is already on their plate, just done badly. Every Indian engineering programme has capstone projects, often invented; Milan replaces that invention with a real, citizen-anchored problem with publication, credit, and CSR funding on the path. The professor gets a real client (the citizen), a real problem statement with context, a real credit chain for the student that survives the project ending, and — if it goes well — a real publication and a potential industry funder. The HOD gets a real claim flow into their lab's capacity, so the lab is not running at 60% utilisation. The department's track record on the platform is public, so it becomes a signal to attract better students and better partners. We are not asking them to do more work; we are replacing a category of work they were already doing — and the replacement comes with a citizen, a credit chain, and a clock.

Q. 22. Why would Tata Steel or an MSME use this instead of running their own CSR directly?

A.

Three reasons, each with a price. **Discovery cost.** Running your own CSR means paying a consultant to find problems you can fund — and they will re-survey the same blocks as the last consultant, badly, because there is no shared registry. Milan is the registry. **Audit cost.** The \$135 CSR report from Milan is one SQL query, used identically by the screen, the CSV and the PDF — so the three can never disagree. "Confirmed / Partly confirmed / Claimed but not confirmed" are three separate counters, never summed. An auditor can defend this document; an auditor cannot defend the alternative. **Reputation cost.** Every contribution is on a public ledger, forever. A company that funds a real fix gets a real, verifiable record; a company that funds a brochure gets a brochure. Industry is also an opt-in `FUNDER` credit edge on the project, so the company is named on the project's public credit chain, not in a press release that ages out. We are not competing with CSR teams; we are giving them the only impact data their auditor, their board and their PR team can all agree on.

Q. 23. What about independent experts who don't want to be associated with their employer?

A.

The role exists. `INDEPENDENT\_INNOVATOR` is in the enum (along with `EXPERT\_PANEL` and `ASSISTED\_SUBMITTER` — the third and fourth roles with no dedicated onboarding screen this cut, declared on a slide). An individual can claim as an independent innovator with personal credit only; their employer is never named, and the UI on `/industry/discover` states that "a legal entity is needed to receive money, not to participate". So an expert can: submit a problem, corroborate, be credited on a project team, and have a public credit page at `/credit/[userId]` — all without any organisational affiliation ever appearing on the chain. The roadmap is a dedicated onboarding screen for the role, with a "claim your credit, anonymously if you prefer" path.

Q. 24. How do you onboard users when the platform is new and empty?

A.

We do not start empty. The seed already has 24 districts (including the JDIP 4.1 reference columns), 263 blocks, 20 organisations (academia, industry, government, NGOs), 25 challenges and a fixed cast of demo accounts that walk the entire lifecycle — three challenges routed to BIT Sindri, one in research, one published with a CSR expression of interest, one citizen-confirmed, one SLA-breached to the bounty board. `pnpm seed:states` runs that cast through the real state machine (not stubs) so the public pages have shape from minute one. For real onboarding, the levers are different for each actor: **citizens** — an assisted-submitter workflow through panchayat offices and NGO partners, plus an IVR/WhatsApp intake seam reserved for the next cut; **HEIs** — pilot with three partner departments (BIT Sindri is in the seed) and let the credit chain + publication path do the word-of-mouth; **industry** — one pilot CSR report built on the data, with the confirmation-gap chart as the headline; **government** — the District Collector at `/gov/gate` and the SLA board at `/gov/sla` are the two surfaces a DC opens daily, and both have a real, structural reason to exist (severity-gated releases, SLA escalations). Empty is a state we do not ship; shape from minute one, real load by month three.

Prototype & Demo

Q. 25. What exactly have you built and what is stubbed?

A.

Built and rehearsed end-to-end: **the full citizen wizard** at `/submit`` (six steps, localStorage drafts, photo blur, EXIF strip, Nominatim place search + GPS + tap-to-pin + district/block dropdown, framing approval, server-side name from session); **the full AI pipeline** ($P0 \rightarrow S1 \parallel S2 \rightarrow S3 \rightarrow S4 \rightarrow S5$) with the provider chain and the offline rules fallback; **the human gate** at `/gov/gate``; **the SLA engine** with the four ladders (WIDEN, OPEN\_ALL, BREACH, GRAND\_CHALLENGE) and the reaper driven by both Vercel Cron and the `/demo`` console; **the provenance ledger** (append-only by trigger, SHA-256 hash chain, in-browser verification at `/ledger``); **the credit + trust system** with the merge-as-signal invariants and the v1.1.0 trust-weighted corroboration; **Emergency Mode** at `/gov/emergency`` (reversible half-time compression, bounded $\times 1.25$ display surge, `ANNUAL_REVIEW`` exempt); **the public pages** (`/challenges``, `/c/[id]``, `/stats``, `/ledger``, `/bounties``, `/artifacts/[id]``, `/flagged``, `/credit/[userId]``, `/track``); **the \$135 CSR export** at `/industry/csr`` and the MoU generator; and **the offline story** — Postgres + MinIO + Mailpit, every AI stage at fallback level 2, 53 of 53 model calls answered offline in the last rehearsed run. **Declared stubs** (on a slide on purpose): IVR + WhatsApp Business intake (the seam is `/api/intake``, not built); SMS + WhatsApp delivery (mock inboxes, DLT registration needed for real); offline PWA sync (localStorage drafts stand in); fine-tuned models (no labelled data, no GPU budget — few-shot + embedding kNN prior instead); full ten-language coverage (Hindi, English, one Santhali sample); live CPGRAMS / JharSewa write API (the exact payload is rendered instead); self-serve institutional onboarding; e-signature and payment rails (the MoU is generated and hashed, never signed); patent/DOI integration and an IP dispute adjudication UI (the prior-art panel is what exists); automatic face / number-plate detection (citizen-driven blur is a stronger privacy claim, and the unblurred original never leaves the device); a separate live emergency response queue and automatic surge-routing (the compression + display surge are built; the rest is not); a PMTiles basemap (markers draw on a blank canvas and say so); block-level boundary geometry (districts are polygon-resolved against real Jharkhand boundaries; blocks remain nearest-centroid within the resolved district). The rule: every stub is named in a slide, not hidden in a footnote.

Q. 26. Can you show us a live challenge being submitted and routed right now?

A.

Yes. From `/demo``, six minutes, no terminal. Sign in as `admin@milan.demo.milan.in`` in one tab, sign in as `dc.gumla@jh.gov.demo.milan.in`` in a second browser profile. The hero challenge, `JH-2026-GUM-0001``, is already in (Sunita Devi's report of the South Koel embankment crack near Basia, in Hindi), and `/demo`` has six buttons that drive it through the real production server actions — never shortcuts. Beat by beat: **0:00** landing page, then `/demo`` health strip, ledger head hash, open deadline count. **0:30** `/submit`` wizard, six steps, photo step saying plainly that faces are not blurred automatically. **1:00** `/demo`` → **Run the pipeline** (~8 s live) — P0, S1, S2, S3, S4, S5, with every receipt visible. **1:30** `/c/JH-2026-GUM-0001`` — open the priority breakdown, point at the arithmetic, every number is clickable. **2:30** DC tab, `/gov/gate`` — the proposal is there, the shortlist is marked "not notified — held at this gate". Back on `/demo``, press **DC confirms the gate** and **HOD claims it**. **3:15** `/demo`` → **Publish the artifact**, open the artifact page, point at the SHA-256 ledgered into the chain. **4:15** `/demo`` → **+21 days** — the reaper runs, the log fills, and the challenge hits the public `/bounties`` board. **5:00** `/ledger`` → **Verify chain** — green. Expand an entry, recompute the hash in the browser, no upload. **5:20** `/demo`` → **Mark implemented**, read Sunita's SMS aloud, then **Citizen confirms** — the impact counter moves. **5:50** `/industry/csr`` — the three separate blocks, the two beneficiary totals that are never summed, the close. If anything goes wrong, `/demo`` → **Reset** restores the demo state in about a second.

Q. 27. Why should we believe the AI pipeline is real and not hardcoded for the demo?

A.

Three independent proofs. **Proof 1 — the receipts.** Every model call — success, failure, cached, offline — writes a row to ``ai_runs`` with its provider, model, fallback level, confidence, latency and input hash. A cache hit still writes a row with ``provider: "cache"``` so the trace never overstates what actually ran. ``/admin/ai-runs`` is the full audit log, with p50/p95 per stage. **Proof 2 — the verifier.** ``/ledger`` → **Verify chain** runs the same hash-chain verification in the browser, recomputing every link from genesis. If anything in the demo were hardcoded, the chain would not verify. It verifies — try changing a byte in any entry and re-running it. **Proof 3 — the offline run.** The offline rehearsal goes through the full pipeline with ``AI_PROVIDER_CHAIN=rules`` and the wifi off. The last rehearsed run: 53 of 53 model calls answered at fallback level 2 (deterministic rules), 4/4 containers healthy, all 13 demo beats passing, no call to Gemini, Groq, Supabase or Resend. The offline run is the strongest possible proof that the pipeline is real: a hardcoded demo would not need a rules fallback, and an AI-wrapper would not be able to run with the wifi off. Finally, the code is the documentation: every stage is in ``lib/ai/pipeline.ts``, every receipt in ``ai_runs``, and ``/admin/triage`` is the live queue where the model hands off to a human at confidence < 0.60 — not a demo feature, a production invariant.

Q. 28. The provenance ledger sounds complex. How does it actually work in the demo?

A.

It is one table, one trigger, and one hash function. **The table** is ``ledger_entries``: ``(seq, content_hash, prev_hash, entry_hash, kind, payload, author_id, created_at)``. **The trigger** is in migration 0002 — a Postgres trigger that ``RAISES`` on any UPDATE or DELETE on the table. Append-only is enforced at the database level, not by application convention. **The hash function** is ``lib/ledger/hash.ts``, pure, runs identically in the browser and on the server. ``canonicalJson()`` sorts object keys at every level, drops undefined/function values, serialises dates as ISO-8601 — so any third party can recompute a hash and get the same answer. ``entryHash = sha256(canonical({seq, contentHash, prevHash, authorId, createdAt}))``. Changing any earlier payload changes its content hash, which changes its entry hash, which is the next entry's ``prevHash`` — verification fails from the tampered entry onward and names its ``seq``. **Appending** (``lib/ledger/append.ts``) runs inside the caller's transaction (a state change, its ledger entry, its SLA deadlines and its outbox event are one atomic fact) and takes a Postgres advisory transaction lock first, so two concurrent appends cannot both read the same tip and fork the chain. In the demo: ``/ledger`` shows the chain paginated, with a **Verify chain** button that walks it from genesis and checks every link. Expand an entry to see the payload and the file-hash calculator — your browser computes the hash, nothing is uploaded. The proof: try tampering with any entry and re-running the verifier — it comes back red and names the sequence number. A verifier that can go red is the only kind worth having.